collect_temp.py — Authoritative Requirements (Current Implementation)

1. Purpose

`collect_temp.py` is a deterministic temperature data acquisition service designed to run continuously on a Raspberry Pi Zero 2.

It:

- Acquires temperature data from DS18B20 sensors on two 1-Wire GPIO buses
- Uses strictly epoch-aligned timing for repeatable sampling
- Buffers records in memory
- Periodically flushes data to:
    - SQLite database
    - MQTT broker (optional, disabled by default)
    - Console debug output
- Is designed for **predictable, repeatable logging**, not resilience or networking complexity

This document is derived directly from the current working source code.

---

2. Hardware & OS Assumptions

- Platform: Raspberry Pi Zero 2
- OS: Linux with 1-Wire support enabled
- Two static overlays configured:

```
/sys/bus/w1/devices/w1_bus_master1
/sys/bus/w1/devices/w1_bus_master2
```

Hardware and bus configuration are assumed static for the duration of program execution.

---

3. Sensors

- Type: DS18B20
- Identified by 1-Wire serial: `28-xxxxxxxxxxxx`

- Sensors may exist on either bus
- Sensors are discovered **once at startup**
- No runtime hot-plug detection
- Invalid readings recorded as `None`

3.1 Sensor Identity Model (Critical)

Sensors are **not keyed by name** internally.

Each sensor is assigned a **column serial**:

```
column_serial = raw_serial with "-" replaced by "_"
28-00000abc1234 → 28_00000abc1234
```

This column serial is used for:

- Record storage
- SQLite column names
- Internal ordering

3.2 Human-Readable Names

A configuration file `temp.cfg` stores:

```
<raw_serial> <name>
```

Unknown sensors are assigned:

```
unknown1, unknown2, ...
```

This file is automatically updated during discovery.

Human names are used only for:

- Debug output
- MQTT payloads

---

4. Discovery Behavior

At startup:

1. Both bus directories are scanned
2. Any entry beginning with `28-` is treated as a sensor
3. Global structures created:

```
sensors[column_serial] = {
```

```
    name,
    bus,
    raw_serial
}

sensor_order = [column_serial, ...]
```

4. `temp.cfg` is rewritten if new sensors are found

4.1 DEBUG Discovery Output

When `DEBUG = True`:

- Bus path printed
- Each sensor printed with assigned name
- Final summary printed

When `DEBUG = False`, discovery is silent.

---

5. Timing Model

Parameter	Value	Meaning
READ_RATE	30 s	Acquisition interval
WRITE_RATE	300 s	Flush interval
CONVERSION_DELAY	0.9 s	Sensor conversion time

5.1 Epoch Alignment

Sampling is aligned to wall-clock epoch boundaries:

```
next = now + (rate - (now % rate))
```

At startup, the program waits until the next alignment boundary.

---

6. Acquisition Cycle

At each aligned epoch:

1. Record trigger epoch timestamp
2. Trigger conversion on each bus via `w1_master_slaves`
3. Sleep `CONVERSION_DELAY`
4. Read every sensor from `/sys/bus/w1/devices/<raw>/w1_slave`

---

7. Temperature Processing

A reading is valid only if:

- CRC line contains `YES`
- Temperature is not `85.0°C`

Valid readings:

1. Converted to Fahrenheit
2. Rounded to nearest 0.1°F
3. Stored as integer: `°F × 10`

Failures return `None`.

---

8. Record Structure (Internal)

Each buffered record:

```
{
  "epoch": <int>,
  "temps": {
      column_serial: value_or_None
  }
}
```

---

9. DEBUG Mode

When enabled:

- Discovery information printed
- Each acquisition prints:

```
MMDDYYYY HH:MM:SS name: value name: value ...
```

- Flush events printed
- Shutdown flush printed

No side effects.

---

10. SQLite Output (DB_WRITE)

10.1 Database

```
/var/lib/logger/database/temperature.db
Table: "Temperature"
```

10.2 Schema

On startup:

```
epoch INTEGER PRIMARY KEY
s<column_serial> INTEGER
```

Example:

```
s28_00000abc1234
```

10.3 Schema Evolution

- New sensors automatically add new columns
- No columns are removed
- Missing readings stored as NULL

10.4 Inserts

Every insert includes all sensor columns.

Epoch uniqueness enforced by PRIMARY KEY.

---

11. MQTT Output (MQTT_WRITE)

Disabled by default.

11.1 Role

MQTT is a **read-only consumer** of the buffer, identical to SQLite.

11.2 Connection

- Broker: `MQTT_BROKER`
- Topic: `MQTT_TOPIC`
- Uses paho-mqtt Callback API v2
- Automatic reconnect on unexpected disconnect

11.3 Payload Translation (Important)

Before publishing, records are translated:

`column_serial → sensor name`

MQTT payload uses human-readable names.

11.4 Payload Format

```
{
  "device": "rpi-zero-2",
  "table": "Temperature",
  "records": [
    {
      "epoch": 1706001234,
      "temps": {
        "LivingRoom": 697,
        "Outdoor": null
      }
    }
  ]
}
```

Entire flush block is published as one payload.

---

12. Buffering & Flush Behavior

- Records appended to shared buffer
- Flush occurs when:

`len(buffer) >= WRITE_RATE / READ_RATE`

- Same block sent to DB and MQTT
- No retry logic
- Errors only visible in DEBUG output

---

13. Shutdown Behavior

On SIGINT / SIGTERM:

1. Loop exits
2. Remaining buffer flushed once
3. MQTT cleanly disconnected (if enabled)

---

14. Configuration Defaults

Setting	Default
DEBUG	True
DB_WRITE	True
MQTT_WRITE	False

---

15. Explicit Non-Goals

The program intentionally does **not** implement:

- Runtime sensor hot-plug
- Retry or persistence on failures
- Dynamic bus configuration
- Cloud connectivity
- Subscription-based MQTT logic

---

16. Architectural Principles

1. Sensors are keyed by **column serial**, not name
2. SQLite columns are `"s" + column_serial`
3. MQTT uses names only as a presentation layer
4. Timing is strictly epoch-aligned
5. Discovery occurs once at startup
6. Design favors deterministic logging over resiliency